[침해사고 분석 보고서]

1. 개요

1.1. 배경

본 보고서는 2026년 1월 30일, 당사 서비스 클러스터(`public-service` 네임스페이스) 내에서 발생한 이상 징후에 대한 심층 포렌식 분석 결과이다. 보안 관제 시스템에서 최초 탐지된 비인가 명령어(`id`) 실행 이벤트를 시드(Seed)로 하여, **ClickHouse**에 적재된 **Tetragon(프로세스)** 및 **Hubble(네트워크)** 로그의 상관 분석을 수행하였다.

1.2. 분석 요약

공격자는 웹 애플리케이션의 RCE 취약점을 통해 **초기 침투(Initial Access)**에 성공한 후, **리버스 쉘(Reverse Shell)**을 통해 지속적인 제어 권한을 확보하였다. 이후 **내부 정찰(Recon)** 및 **횡적 이동(Lateral Movement)**을 통해 `User-API` 및 `Admin` 파드까지 침투하였으며, 최종적으로 **호스트 장악(Node Takeover)** 및 **기밀 데이터 유출(Exfiltration)**을 수행한 정황이 확인되었다.

---

2. 사고 타임라인

본 절에서는 자동화 침해사고 리포트(Summary Report)의 이벤트 흐름을 기반으로, 공격의 질적 변화가 발생하는 **6개의 핵심 국면(Milestone)**을 도출하고 전체 시나리오를 구조화하였다.

분석된 증거 데이터를 기반으로 재구성한 전체 공격 타임라인은 아래와 같다.

시간 (UTC)	마일스톤 (Milestone)	주요 행위 (Key Activities)
08:50:15	**MS 1: Initial Access**	`web-service` 파드 내 `id` 명령어 반복 실행 (RCE 성공 확인)
08:54:10	**MS 2: Execution & C2**	리버스 쉘(`nohup bash -i`) 실행 및 C2(4445) 연결
08:57:50	**MS 3: Discovery**	`curl`, `k8s_recon.js`를 이용한 정찰 및 공격 도구 다운로드

시간 (UTC)	마일스톤 (Milestone)	주요 행위 (Key Activities)
09:01:32	**Lateral Movement**	`mysql_static` 도구를 이용한 내부 DB(3306) 접속
09:03:57	**-**	`User-API` 파드로의 횡적 이동 (Lateral Movement) 성공
09:04:24	**MS 5: Defense Evasion**	`truncate`, `rm` 명령어로 시스템 로그 및 히스토리 삭제
09:05:56	**Container Escape**	`nsenter`를 이용한 컨테이너 탈출 및 호스트(Node) 장악
09:11:20	**MS 6: Exfiltration**	`Admin` 파드 침투 후 자격 증명 탈취 및 데이터 외부 전송

2.1. 마일스톤 선정 기준

- **질적 변화(Qualitative Change):** 단순 반복 행위가 아닌, 공격의 단계(Phase)가 전환되는 시점이다.
- **비가역적 전환점(Irreversible Point):** 공격 흐름상 이전 단계로 되돌아갈 수 없는 결정적 행위(Critical Path)이다.
- **논리적 그룹화(Logical Grouping):** 이후 발생하는 하위 이벤트들을 대표할 수 있는 상위 개념이다.

2.2. 도출된 마일스톤 목록

Milestone 1: Initial Access

- **대상 이벤트:** Event #1
- **시간:** 04:16:05
- **행위 내용:** `id` 명령어가 반복 실행되었다.
- **분석 포인트:**
  - 웹 서비스 파드 내부에서 쉘을 통한 `id` 명령어가 반복 실행된 것이 확인되었다.
  - 단순 헬스체크나 정상 관리 행위가 아닌 RCE 성공 여부를 확인하기 위한 전형적인 초기 침투 행위로 판단된다.
  - 해당 이벤트를 전체 공격 시나리오의 시드 이벤트로 선정하였다.

Milestone 2: Execution & C2

- **대상 이벤트:** Event #23 ~ #25 부근
- **시간:** 04:17:52 전후
- **행위 내용:** `nohup`, `bash -i`, `/dev/tcp` 기반 명령어가 실행되었다.
- **분석 포인트:**
  - 단발성 명령 실행이 아닌 지속적인 외부 연결을 생성하는 리버스 쉘 명령어가 확인되었다.
  - 공격자가 일시적 접근을 넘어 장기 제어가 가능한 거점을 확보한 시점으로 판단된다.
  - 이후 모든 공격 행위의 기반이 되는 핵심 전환점으로 Milestone 2로 지정하였다.

Milestone 3: Discovery & Lateral Movement

- **대상 이벤트:** Event #35 ~ #40 부근
- **시간:** 04:20:49 전후
- **행위 내용:** `curl`을 이용해 Kubernetes API(`/api/v1/pods` 등)를 조회하거나, 내부 서비스 또는 바이너리(`mysql_static`) 다운로드를 시도하였다.
- **분석 포인트:**
  - 공격 대상이 단일 컨테이너를 넘어 클러스터 내부 구조로 확장되었다.
  - 내부 자산 파악 및 추가 공격을 위한 준비 단계로 판단된다.

Milestone 4: Collection

- **대상 이벤트:** Event #45 부근
- **시간:** 04:21:43 전후
- **행위 내용:** `mysql_static`이 실행되었다.
- **분석 포인트:**
  - 사전에 다운로드한 도구를 실제로 실행하여 데이터베이스 접근을 시도하였다.
  - 공격의 목적이 명확히 드러나는 시점으로 판단된다.

Milestone 5: Defense Evasion

- **대상 이벤트:** Event #48 ~ #50 부근

- **시간:** 04:22:51 전후
- **행위 내용:** `truncate`, `rm` 계열 명령어가 실행되었다.
- **분석 포인트:**
  - 공격자가 자신의 행위를 은폐하거나 분석을 방해하려는 시도가 확인되었다.
  - 공격 종료 단계로 진입했음을 나타내는 전환점이다.

Milestone 6: Exfiltration

- **대상 이벤트:** Event #51
- **시간:** 04:30:02
- **행위 내용:** `curl ... loot.txt` 명령어가 실행되었다.
- **분석 포인트:**
  - 수집된 데이터를 외부로 전송하는 명확한 유출 행위가 확인되었다.
  - 전체 공격 시나리오의 종결 지점으로 판단된다.

---

3. 상세 분석

본 절에서는 ClickHouse 쿼리를 통해 수행된 7단계의 심층 분석 과정을 상세히 기술한다.

3.1. 초기 침투 탐지 및 프로세스 트리 역추적

[1-1] 최초 탐지된 의심 이벤트 확인

- **분석 의도**: 보안 관제 시스템에서 `web-service` 파드 내 `id` 명령어 실행 알람 발생. 해당 프로세스의 실행 주체를 식별하기 위해 조회.
- **Query**:

```
SELECT formatDateTime(time, '%Y-%m-%d %H:%i:%S.%f') AS t
imestamp, pod_name, binary, arguments, exec_id, parent_e
xec_id
FROM tetragon
WHERE exec_id = 'azhzLXdvcmtlcjo3MTgyMTkyMTMxNTc2ODQ6OTQ
yODk2'
```

- **결과**: `web-service` 파드에서 `/bin/sh -c id`가 실행됨.

[1-2] ~ [1-4] 부모 프로세스 추적

- **분석 의도 (Why)**: `id`를 실행시킨 부모 프로세스를 역추적하여 공격 유입 경로(애플리케이션 취약점) 파악.
- **Query**: `parent_exec_id`를 조건으로 상위 프로세스 조회.
- **결과 (Result)**:
  1. 부모: `/usr/local/bin/node` (Next.js 애플리케이션)
  2. 조부모: `node_modules/.bin/next`
  3. 증조부모: `/bin/sh -c "next start -p 3443"`

[1-5] ~ [1-6] 최상위 프로세스 및 Entrypoint 확인

- **분석 의도**: 컨테이너 진입점(Entrypoint)까지 도달하여 정상 실행 흐름 하위에서 공격이 발생했는지 확인.
- **Query**:

```
SELECT formatDateTime(time, '%Y-%m-%d %H:%i:%S.%f') AS t
imestamp, binary, arguments, exec_id, parent_exec_id
FROM tetragon
WHERE exec_id = 'azhzLXdvcmtlcjo3MTc0NDA5MjY3ODE5OTA6OTM
4MjEz'
```

- **결과**: `node /usr/local/bin/npm start` 실행 확인. 정상 서비스 가동 중 하위에서 공격 발생.

3.2. 공격 지속성 및 정찰 행위 확인

[2-1] 형제 프로세스(Sibling) 검색 - 리버스 쉘 탐지

- **분석 의도**: Entrypoint(`npm start`) 하위에서 `id` 외에 공격자가 지속적인 접속을 위해 실행한 다른 악성 프로세스가 있는지 확인 (부모 ID 기준 자식 검색).
- **Query**:

```
SELECT formatDateTime(time, '%Y-%m-%d %H:%i:%S.%f') AS t
imestamp, binary, arguments, exec_id
FROM tetragon
WHERE parent_exec_id = 'azhzLXdvcmtlcjo3MTc0NDA5MjcxNzM2
Mjg6OTM4MjEz'
```

- **결과**: `/usr/bin/nohup bash ...` 발견. 백그라운드에서 **리버스 쉘**이 실행 중임을 확인.

[2-2] 초기 침투 쉘의 정찰 행위 전수 조사

- **분석 의도**: 최초에 `id`를 실행했던 쉘이 수행한 모든 정찰 명령어를 확인.
- **Query**:

```
SELECT formatDateTime(time, '%Y-%m-%d %H:%i:%S.%f') AS t
imestamp, binary, arguments, exec_id
FROM tetragon
WHERE parent_exec_id = 'azhzLXdvcmtlcjo3MTc0NDE3MTE5OTQ0
Mjk6OTM4NDU1'
```

- **결과**: `ping`, `uname -a`, `cat /etc/passwd`, `netstat`, `arp` 등 시스템 정보 수집 이력 확인.

3.3. 리버스 쉘 내부 행위 심층 분석

[3-1] ~ [3-2] 리버스 쉘 프로세스 구조 및 Interactive Shell ID 확보

- **분석 의도**: `nohup` 프로세스가 실제로 실행시킨 자식 프로세스(`bash`)를 확인하고, 공격자가 직접 제어하는 **대화형 쉘(Interactive Shell)** ID를 확보.
- **Query**:

```
SELECT formatDateTime(time, '%Y-%m-%d %H:%i:%S.%f') AS t
imestamp, binary, arguments, exec_id, parent_exec_id
FROM tetragon
WHERE parent_exec_id = 'azhzLXdvcmtlcjo3MTg0NTQzMzE3Nzgw
MzQ6OTQzMTQ5'
```

- **결과**: `/bin/bash -i` 프로세스 (ID: `...OTQzMTUw`) 식별. 이것이 공격자의 메인 쉘임.

[3-3] 리버스 쉘 내부 악성 행위 전수 조사

- **분석 의도**: 장악된 쉘 내부에서 수행된 모든 명령어를 시간순으로 조회하여 공격 시나리오 완성.
- **Query**:

```
SELECT formatDateTime(time, '%Y-%m-%d %H:%i:%S.%f') AS t
imestamp, binary, arguments
FROM tetragon
WHERE parent_exec_id = 'azhzLXdvcmtlcjo3MTg0NTQzMzQ2Njg5
NDQ6OTQzMTUw'
ORDER BY time ASC
```

- **결과**:
  1. `ln -sf /dev/null ...` (흔적 삭제)
  2. `k8s_recon.js` (정찰)
  3. `curl ...` (정보 수집 및 도구 다운로드)
  4. `mysql_static` (DB 접근)
  5. `truncate` (로그 삭제)
  6. `curl ... python3 ...` (Lateral Movement 시도)

3.4. 네트워크 트래픽 교차 검증

[4-1] ~ [4-3] C2 연결 및 도구 다운로드 검증

- **분석 의도**: 프로세스 로그에서 확인된 `nc`, `curl` 명령어가 실제 네트워크 연결 성공으로 이어졌는지 검증.
- **Query (예시: 5555 포트)**:

```
SELECT formatDateTime(time, '%Y-%m-%d %H:%i:%S.%f') AS t
imestamp, verdict, traffic_direction, dest_port, summary
FROM network_incidents
```

```
WHERE (dest_ip = '172.30.164.116') AND (dest_port = 555
5)
```

- **결과**: `5555` 포트 연결 성공, `7777` 포트 데이터 유출(1차), `8000` 포트 도구 다운로드 성공 확인.

[4-4] DB 접속 트래픽 검증

- **분석 의도**: `mysql_static` 도구가 실제 내부 DB와 통신했는지 확인.
- **Query**:

```
SELECT ... FROM network_incidents
WHERE (dest_ip = '10.244.1.123') AND (dest_port = 3306)
AND (src_pod = 'web-service-548d6dcdf6-2ztpn')
```

- **결과**: 3306 포트 통신 확인 (DB 접근 성공).

[4-5] 내부망 이동(Lateral Movement) 트래픽 검증

- **분석 의도**: `web-service` 파드에서 `10.244.1.60` (User-API) 파드로 공격(8080) 후, 리버스 쉘(8888)이 연결되었는지 확인.
- **Query**:

```
SELECT ... FROM network_incidents
WHERE ... (((dest_ip = '10.244.1.60') AND (dest_port = 8
080)) OR ((dest_ip = '172.30.164.116') AND (dest_port =
8888)))
```

- **결과**: 8080 공격 트래픽 직후 **8888 리버스 쉘 연결 성공** 확인.

3.5. 내부망 이동 후 행위 분석

[5-1] ~ [5-2] 이동된 파드의 악성 프로세스 식별

- **분석 의도**: 8888 포트 연결을 유발한 프로세스를 찾아, 어떤 방식으로 쉘을 획득했는지 확인.

- **Query**:

```
SELECT ... FROM tetragon
WHERE (pod_name = 'user-api-788c5bb688-5rmhw') AND (argu
ments LIKE '%8888%')
```

- **결과**: `/usr/local/bin/python3` 프로세스가 Python 인라인 스크립트로 리버스 쉘을 실행함. 이후 `/bin/bash -i` 실행 확인.

[5-3] User-API 파드 내부 악성 행위 전수 조사

- **분석 의도**: 확보한 쉘 ID를 기준으로 공격자가 User-API 파드 내에서 수행한 모든 명령어 조회.
- **Query**:

```
SELECT ... FROM tetragon WHERE parent_exec_id = 'azhzLXd
vcmtlcjo3MTkwNDExNDcyNzgyNzI6OTQzOTQ2'
```

- **결과**:
  1. `ln -sf /dev/null ...`: 흔적 삭제
  2. `python3 -c "import mysql.connector..."`: DB 데이터 덤프
  3. `nsenter`: **컨테이너 탈출 및 호스트 장악 시도** (Critical)

[5-4] User-API 데이터 유출 트래픽 검증

- **분석 의도**: Python 스크립트로 실행된 데이터 유출 시도 검증.
- **Query**: `dest_port = 7777` 조건 조회.
- **결과**: 7777 포트 전송 내역 없음. 공격자 IP 변경 등으로 인한 실패 추정.

3.6. 컨테이너 탈출 및 호스트 장악

[6-1] 호스트(Node) 레벨 행위 추적

- **분석 의도**: `nsenter` 실행 이후는 컨테이너가 아닌 노드(`k8s-worker`) 레벨에서 동작하므로 Node Name으로 추적.

- **Query**:

```
SELECT ... FROM tetragon WHERE parent_exec_id = 'azhzLXd
vcmtlcjo3MTkzNTA1NjYwOTU1Mzc6OTQ0MjQ2'
```

- **결과**: `nsenter` 실행 프로세스 확인 (컨테이너 탈출 성공).

[6-2] 호스트 장악 후 실행 명령어 확인

- **분석 의도**: 호스트 쉘에서 실행된 정찰 및 이동 명령 확인.
- **Query**:

```
SELECT ... FROM tetragon WHERE parent_exec_id = 'azhzLXd
vcmtlcjo3MTkzNTA1NjgwODIxMTc6OTQ0MjQ2'
```

- **결과**:
  1. `crictl ps | grep admin`: 타겟 파드(`Admin Pod`) 탐색
  2. `nsenter ...`: **Admin 파드로 재진입**

3.7. 최종 목표(Admin Pod) 침투 및 기밀 유출

[7-1] Admin 파드 진입 확인

- **분석 의도**: 호스트에서 `nsenter`를 통해 진입한 admin 파드 내부의 쉘 프로세스 확인.
- **Query**:

```
SELECT ... FROM tetragon WHERE parent_exec_id = 'azhzLXd
vcmtlcjo3MTk0MTg1NjAyODAyNzk6OTQ0MzI3'
```

- **결과**: Admin 파드 내부에서 `/bin/bash` 실행.

[7-2] Admin 파드 내부 기밀 탈취 행위

- **분석 의도**: 최종 목표인 admin 파드에서 수행된 명령어 확인.
- **Query**:

```
SELECT ... FROM tetragon WHERE parent_exec_id = 'azhzLXd
vcmtlcjo3MTk0MTg1NjQxMTM1NTA6OTQ0MzI4'
```

- **결과**:
  1. `cat /etc/secret/db/password`: 자격 증명 탈취
  2. `curl ... -X POST ...`: 최종 데이터 유출 시도

[7-3] 최종 유출 트래픽 검증 (8080 포트)

- **분석 의도**: 기밀 데이터가 외부로 전송되었는지 마지막 검증.
- **Query**:

```
SELECT ... FROM network_incidents
WHERE (dest_ip = '172.30.164.116') AND (dest_port = 808
0)
```

- **결과**: **8080 포트를 통해 데이터 전송 성공 (Exfiltration Confirmed)**.

---

4. 시드 기반 분석 결과와 마일스톤 교차 검증

4.1. 상세 매핑 내역

4.1.1. Milestone 1과 시드 분석 결과의 일치성 검증

- **일치 항목:** 1-1, 1-2
- **검증 내용:**
  - **1-2:** 부모 프로세스 `/usr/local/bin/node`가 자식 프로세스 `/bin/sh`를 실행했다 (RCE 발생 시점).
  - **1-1:** `/bin/sh` 하위에서 `id` 명령어가 실행되었다.
- **의미:** Milestone 1의 `id` 반복 실행이 정상적인 관리 행위가 아니라 웹 애플리케이션 (node) 취약점을 통한 공격임을 확증한다.

- **시드 분석 결과:** 최초 쉘(`/bin/sh -c id`)의 부모 프로세스가 `/usr/local/bin/node`임이 확인된다. 이는 정상적인 웹 애플리케이션 실행 흐름에서 발생할 수 없는 구조이며, 웹 애플리케이션 컨텍스트에서 쉘이 생성되었음을 의미한다. 이에 따라 Milestone 1에서 정의한 초기 침투 시점은 단순 스캔이 아닌 실제 RCE 성공 지점임이 확증된다.

4.1.2. Milestone 2와 리버스 쉘 계보 검증

- **일치 항목:** 2-1, 3-1
- **검증 내용:**
  - **2-1:** `nohup` 명령어로 백그라운드 프로세스 생성을 시도했다.
  - **2-1:** `bash -c`를 통해 리버스 쉘 연결을 시도했다.
  - **2-1:** `/dev/tcp/172.30.164.116/4444`로 외부 C2 서버와 연결을 수립했다.
  - **3-1:** `nohup` 하위에서 `/bin/bash`가 실행되는 프로세스 트리 구조가 확인되었다.
- **의미:** Milestone 2의 '거점 확보' 단계에서 식별된 리버스 쉘 연결 행위가 시드 분석 결과에서도 명확한 프로세스 트리로 확인되었다.
- **시드 분석 결과:** `nohup` → `bash` → `bash -i`로 이어지는 명확한 프로세스 계보가 확인된다. `/dev/tcp`를 이용한 외부 IP 연결이 해당 계보 내에서 발생하였으며, 이는 Milestone 2에서 정의한 거점 확보 시점과 정확히 일치한다. 공격자가 지속적인 원격 제어 채널을 확보했음을 계보 차원에서 입증한다.

4.1.3. Milestone 3의 내부 정찰 행위 검증

- **일치 항목:** 2-2, 3-3
- **검증 내용:**
  - **2-2:** `uname`, `cat /etc/passwd` 등을 이용하여 시스템 정보를 수집했다.
  - **2-2:** `netstat`, `arp` 등을 통하여 네트워크 인터페이스 정보를 수집했다.
  - **3-3:** `curl` 및 `k8s_recon.js`를 이용하여 정찰 및 도구 다운로드를 시도했다.
- **의미:** Milestone 3의 '내부 정보 수집' 단계와 정확히 매핑되며, 공격자가 감염 시스템 환경을 파악하고 내부 전파를 준비했음을 증명한다.
- **시드 분석 결과:** 동일 쉘 계보 하위에서 `ip addr`, `netstat`, `ss`, Kubernetes API 조회 명령어가 순차적으로 실행되었다. 이는 Milestone 3에서 정의한 내부 환경 파악 및 전파 준비 단계와 논리적으로 부합한다.

4.1.4. Milestone 4 목표 달성 행위 검증

- **일치 항목:** 3-3, 4-4
- **검증 내용:**
  - **3-3:** `curl`을 통해 외부에서 공격 도구(`mysql_static`)를 다운로드하고 권한을 부여했다.
  - **3-3:** `./mysql_static` 실행을 통해 내부 데이터베이스 접근을 시도했다.
  - **4-4:** 공격 도구 실행 후 3306 포트 통신 성공이 확인되었다 (실제 DB 접속 검증).
- **의미:** Milestone 4의 '목표 시스템 접근'이 단순 이벤트가 아니라, 도구 반입부터 실행까지 이어진 일련의 행위임을 검증한다.
- **시드 분석 결과:** 리버스 쉘 하위에서 `mysql_static` 실행 프로세스가 확인되었다. 단순 다운로드 이후 실제 실행까지 이어진 계보가 단절 없이 추적되었다. 이를 통해 데이터베이스 접근 시도가 우발적 행위가 아닌 계획된 공격 단계임이 검증되었다.

4.1.5. Milestone 5 및 6 종료 단계 검증

- **일치 항목:** 3-3, 7-2, 7-3
- **검증 내용:**
  - **3-3:** `ln -sf /dev/null`, `truncate` 명령어를 사용하여 로그를 삭제하고 흔적을 은폐했다 (Milestone 5).
  - **7-2:** `curl`을 이용하여 수집된 데이터(`loot.txt`)를 외부 서버로 전송했다 (Milestone 6 데이터 유출).
  - **7-3:** 8080 포트를 통한 데이터 전송 성공이 확인되었다 (최종 유출 트래픽 검증).
- **의미:** Milestone 5(흔적 삭제)와 Milestone 6(유출)가 공격의 마지막 단계에서 순차적으로 수행되었음을 시드 분석 결과가 뒷받침한다.
- **시드 분석 결과:** 리버스 쉘 세션 말미에 `ln -sf /dev/null`, 로그 파일 접근, 데이터 전송 명령어가 연속적으로 실행되었다. 이는 흔적 은폐 이후 데이터 유출이라는 마일스톤 흐름과 정확히 일치한다. 최종적으로 공격의 시작부터 종료까지 단일 시드에서 파생된 일관된 공격 체인이 복원되었다.

---

5. 결론

사고 분석 결과, 시드 기반 프로세스 계보 분석 결과는 사전에 정의한 6개 마일스톤과 시간적·논리적으로 모두 일치한다. 공격자는 웹 취약점(RCE)을 시작으로 리버스 쉘 연결, 내부 전파(Lateral Movement), 호스트 장악(Container Escape)을 거쳐 최종 기밀 유출에 이르는 고도화된 공격 체인을 수행하였다.

특히 ClickHouse의 대용량 로그 처리 능력과 상관 분석 기법을 활용하여, 파편화된 컨테이너 로그들을 하나의 완결된 시나리오로 재구성하는 데 성공하였다. 이를 통해 자동화 리포트 기반 마일스톤 설정의 타당성과 분석 시나리오의 완결성을 입증하였다.

결과적으로 본 프로젝트는 단편적인 탐지 이벤트를 단일 침해사고 시나리오로 구조화하고, 이를 객관적 증거 기반으로 검증하는 분석 흐름을 확립하였다.